

Question 1 — IoT Sensor Dashboard

Problem 1 — Sensor Reading Classifier

Source code:

```
#include<iostream>

using namespace std;

int main(){

    double range;

    cout<<"Enter the Range of temperature: ";

    cin>>range;

    double F=(range*9/5)+32;

    cout<<"Temperature : "<<range<<" / "<<F<<endl;

    int statusCode;

    if(range<0){

        statusCode= -1;

    }else if(range>=0 && range<=29){

        statusCode= 0;

    }else if(range>=30 && range<=44){

        statusCode= 1;

    }else if(range>=45 && range<=59){

        statusCode= 2;

    }else{

        statusCode= 3;

    }

    switch(statusCode){

    case -1:

        cout<<"Status : SENSOR_ERROR"<<endl;

        cout<<"Action : Sensor fault — check wiring"<<endl;

        break;

    case 0:
```

```

        cout<<"Status : NORMAL"<<endl;

        cout<<"Action : No action required"<<endl;

        break;

    case 1:

        cout<<"Status : WARNING"<<endl;

        cout<<"Action : Alert sent to supervisor"<<endl;

        break;

    case 2:

        cout<<"Status : CRITICAL"<<endl;

        cout<<"Action : Cooling system triggered"<<endl;

        break;

    default:

        cout<<"Status : SHUTDOWN"<<endl;

        cout<<"Action : Emergency shutdown initiated"<<endl;

    }

    cout<<"Reading : "<< (range>=25?"Above Average" : "Below Average")<<endl;

    return 0;

}

```

```

Enter the Range of temperature: 41
Temperature : 41 / 105.8
Status : WARNING
Action : Alert sent to supervisor
Reading : Above Average

```

Problem 2 — Sensor Log Buffer

Source code:

```
#include<iostream>

using namespace std;

int main(){
    int reading;

    cout<<"Reading Entered: ";

    cin>>reading;

    double arr[reading];

    for(int i=0;i<reading;i++){
        cin>>arr[i];
    }

    cout<<"Valid Reading : ";

    int count=0;

    double min=0;

    double max=0;

    bool check=true;

    int Normal=0;

    int Warning=0;

    int Critical=0;

    int Shutdown=0;

    for(int i=0;i<reading;i++){
        if(arr[i]<0){
            count++;

            continue;
        }

        cout<<arr[i]<<" ";

        if(check){
            min=arr[i];
```

```

        max=arr[i];
        check=false;
    } else {
        if(arr[i]<min){
            min=arr[i];
        }
        if(arr[i]>max){
            max=arr[i];
        }
    }
    if(arr[i]<=29){
        Normal++;
    } else if (arr[i]<=44){
        Warning++;
    } else if (arr[i]<=59){
        Critical++;
    } else {
        Shutdown++;
    }
}

cout<<endl;
cout<<"Skipped (errors) : "<<count<<endl;
cout<<"First CRITICAL : ";
for(int i=0;i<reading;i++){
    if(arr[i]>45){
        cout<<"Index "<<i<<" ->"<<arr[i]<<endl;
        break;
    }
}
}

```

```

        cout<<"Min: "<<min<<" Max: "<<max<<" Avg: "<<(min+max)/2<<endl;

        cout<<"Normal:"<<Normal<<" Warning:"<<Warning<<" Critical:"<<Critical<<"
Shutdown:"<<Shutdown;

        return 0;
}

```

Output:

```

Reading Entered: 8
22.1 31.5 46.0 28.0 50.2 10.0 38.0 -9
Valid Reading : 22.1 31.5 46 28 50.2 10 38
Skipped (errors) : 1
First CRITICAL : Index 2 ->46
Min: 10 Max: 50.2 Avg: 30.1
Normal:3 Warning:2 Critical:2 Shutdown:0

```

Problem 3 — Building Sensor Grid

Source code:

```

#include<iostream>

using namespace std;

int main(){

    double arr[3][3];

    cout<<"Enter temperatures for 3 floors and 3 rooms:";

    for(int i=0;i<3;i++){

        for(int j=0;j<3;j++){

            cin>>arr[i][j];

        }

    }

    cout << "    Room1  Room2  Room3" << endl;

    for(int i=0;i<3;i++){

        cout<<"Floor "<<i+1<<" : ";

        for(int j=0;j<3;j++){

            cout<<arr[i][j]<<" ";

        }

    }
}

```

```

        cout<<endl;
    }
    double hottest=0;
    int hottestfloor=0;
    int hottestroom=0;
    int warningcount=0;
    for(int i=0;i<3;i++){
        for(int j=0;j<3;j++){
            if(arr[i][j]>hottest){
                hottest=arr[i][j];
                hottestfloor=i;
                hottestroom=j;
            }
            if(arr[i][j]>=30){
                warningcount++;
            }
        }
    }

    cout<<"Hottest Room : Floor "<<hottestfloor<<" , Room "<<hottestroom<<" ->
"<<hottest<<endl;

    double highestAverage=0;
    int highestFloor=0;
    for(int i=0;i<3;i++){
        {
            double sum=0;
            for(int j=0;j<3;j++){
                {
                    sum=sum+arr[i][j];
                }
            }
        }
    }

```

```

        double avg=sum/3;

        if (avg>highestAverage)
        {
            highestAverage=avg;
            highestFloor=i;
        }
    }

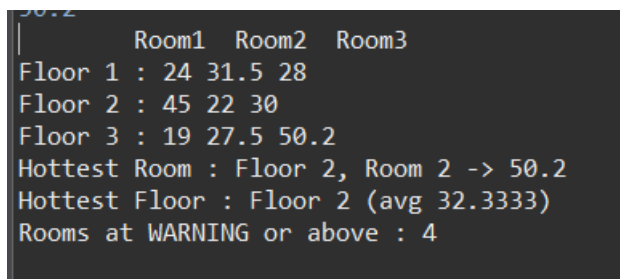
    cout<<"Hottest Floor : Floor "<<highestFloor+1<<" (avg
"<<highestAverage<<)"<< endl;

    cout<<"Rooms at WARNING or above : "<< warningcount << endl;

    return 0;
}

```

Output:



```

50.2
|      Room1  Room2  Room3
Floor 1 : 24 31.5 28
Floor 2 : 45 22 30
Floor 3 : 19 27.5 50.2
Hottest Room : Floor 2, Room 2 -> 50.2
Hottest Floor : Floor 2 (avg 32.3333)
Rooms at WARNING or above : 4

```

Problem 4 — Startup Configuration via CLI

Source code:

```

#include<iostream>

#include <cstdlib>

using namespace std;

int main(int argc,char *argv[]){
    if(argc<4){
        cout<<"Usage : ./sensor_monitor"<<endl;
        cout<<"Error : Missing arguments."<<endl;
        return 1;
    }
}

```

```

int warn=stoi(argv[1]);
int critical=stoi(argv[2]);
int num_reading=stoi(argv[3]);
if(warn>=critical){
    cout<<"Error : warn must be less than critical."<<endl;
    return 1;
}

if(num_reading<1||num_reading>500){
    cout<<"Error : num_reading must be between 1 and 500."<<endl;
    return 1;
}

cout<<"Config :
Warn="<<warn<<"Critical="<<critical<<"Readings="<<num_reading<<endl;

int normal = 0;
int warning = 0;
int criticalcount = 0;
int shutdown = 0;
for(int i=0;i<num_reading;i++){
    int temperature=rand()%70;
    if(temperature<warn){
        normal++;
    }else if(temperature<critical){
        warning++;
    }else if(temperature < 60){
        criticalcount++;
    }else{
        shutdown++;
    }
}

```

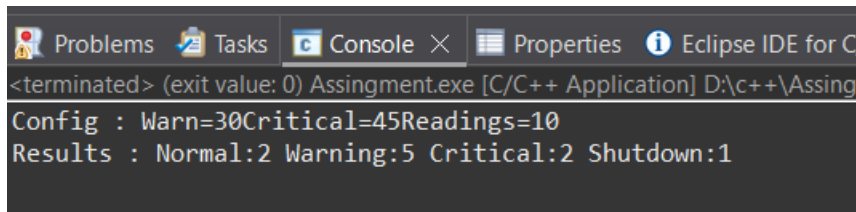
```

        cout << "Results : Normal:" << normal<< " Warning:" << warning<< " Critical:" <<
criticalcount<< " Shutdown:" << shutdown << endl;

        return 0;
}

```

Output:



The screenshot shows the Eclipse IDE console window. The title bar includes tabs for Problems, Tasks, Console, Properties, and Eclipse IDE for C/C++. The console output displays the program's configuration and results:

```

<terminated> (exit value: 0) Assingment.exe [C/C++ Application] D:\c++\Assing
Config : Warn=30Critical=45Readings=10
Results : Normal:2 Warning:5 Critical:2 Shutdown:1

```

Question 2 — Embedded Memory Manager & Signal Processor

Problem 1 — The Bug in Sensor Recalibration

Source code:

```

#include<iostream>

using namespace std;

void resetSensorPairV1(int a,int b){

    int temp=a;

    a=b;

    b=temp;

}

void resetSensorPairV2(int &a,int &b){

    int temp=a;

    a=b;

    b=temp;

}

void resetSensorPairV3(int *a,int*b){

    int temp=*a;

    *a=*b;

    *b=temp;

}

```

```

int main(){
    int a=55;
    int b=12;
    cout<<"--- V1: Call by Value ---"<<endl;
    cout<<"Before : A="<<a<<" B="<<b<<endl;
    resetSensorPairV1(a,b);
    cout<<"After : A="<<a<<" B="<<b<<endl;
    cout<<"--- V2: Call by Reference ---"<<endl;
    cout<<"Before : A="<<a<<" B="<<b<<endl;
    resetSensorPairV2(a,b);
    cout<<"After : A="<<a<<" B="<<b<<endl;
    cout<<"--- V3: Call by Pointer ---"<<endl;
    cout<<"Before : A="<<a<<" B="<<b<<endl;
    resetSensorPairV3(&a,&b);
    cout<<"After : A="<<a<<" B="<<b<<endl;
    return 0;
}

```

Output:

```

--- V1: Call by Value ---
Before : A=55 B=12
After : A=55 B=12
--- V2: Call by Reference ---
Before : A=55 B=12
After : A=12 B=55
--- V3: Call by Pointer ---
Before : A=12 B=55
After : A=55 B=12

```

Problem 2 — Signal Processing Pipeline

Source code:

```
#include<iostream>

#include<cmath>

using namespace std;

double computeRMS(double* signal,int n){
    double sum=0;
    for(int i=0;i<n;i++){
        double current =*(signal+i);// to get the current element instead of using arr[i]
        sum=sum+(current*current);
    }
    double mean=sum/n;
    return sqrt(mean);
}

void normalise(double* signal, int n){
    double maxAbs=0;
    for(int i=0;i<n;i++){
        double current=*(signal+i);
        double currentAbs=abs(current);
        if(currentAbs>maxAbs){
            maxAbs=currentAbs;
        }
    }
    for(int i=0;i<n;i++){
        *(signal+i)=*(signal+i)/maxAbs;
    }
}

int countZeroCrossings(double* signal, int n){
    int count = 0;
```

```

for(int i=0;i<n-1;i++){
    double current=*(signal+i);
    double next=*(signal+i+1);
    if(current>0 && next<0 || current <0 && next>0){
        count++;
    }
}

return count;
}

void applyGain(double* signal,int n,double gainFactor){
    for(int i=0;i<n;i++){
        *(signal+i)=*(signal+i)*gainFactor;
    }
}

int main(){
    double arr[7]={0.5, -1.2, 0.8, -0.3, 1.0, -0.9, 0.1};
    int n=7;
    double rms=computeRMS(arr,n);
    cout<<"RMS: "<<rms<<endl;
    int crossings=countZeroCrossings(arr,n);
    cout<<"Zero Crossings="<<crossings<<endl;
    cout << "Before Normalise : ";
    for(int i = 0; i < n; i++){
        cout << arr[i] << " ";
    }
    cout << endl;
    normalise(arr, n);
    cout << "After Normalise : ";
    for(int i = 0; i < n; i++){

```

```

        cout << arr[i] << " ";
    }

    cout << endl;

    applyGain(arr, n, 2.0);

    cout << "After Gain : ";

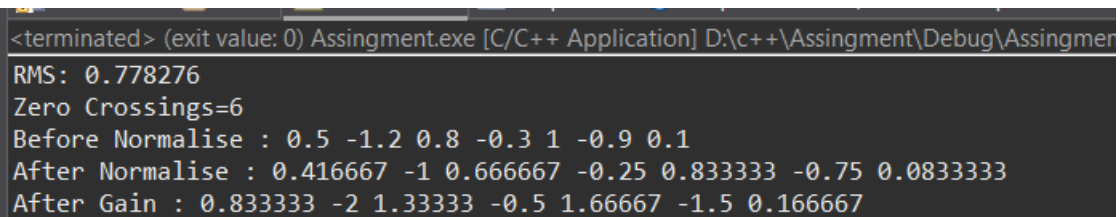
    for(int i = 0; i < n; i++){
        cout << arr[i] << " ";
    }

    cout << endl;

    return 0;
}

```

Output:



```

<terminated> (exit value: 0) Assingment.exe [C/C++ Application] D:\c++\Assingment\Debug\Assingmen
RMS: 0.778276
Zero Crossings=6
Before Normalise : 0.5 -1.2 0.8 -0.3 1 -0.9 0.1
After Normalise : 0.416667 -1 0.666667 -0.25 0.833333 -0.75 0.0833333
After Gain : 0.833333 -2 1.33333 -0.5 1.66667 -1.5 0.166667

```

Problem 3 — Hardware Register Access

Source code:

```

#include<iostream>

using namespace std;

int main(){

    int statusReg = 0b10110001;

    int controlReg = 0b00000000;

    int dataReg = 0b11001010;

    const int* regPtr1 = &statusReg;

    cout << "--- regPtr1 : const int* ---" << endl;

    cout << "Status Register = " << *regPtr1 << endl;
}

```

```

// *regPtr1 = 10;

// ERROR: Cannot modify the value through a pointer-to-const.

regPtr1 = &dataReg;

// Repointing is allowed because the pointer itself is not const.

int* const regPtr2 = &controlReg;

cout << "--- regPtr2 : int* const ---" << endl;

*regPtr2 = 25;

// Value can be changed because the pointed data is not const.

cout << "Control Register = " << *regPtr2 << endl;

// regPtr2 = &dataReg;

// ERROR: Cannot change the address because the pointer itself is const.

const int* const regPtr3 = &statusReg;

cout << "--- regPtr3 : const int* const ---" << endl;

cout << "Status Register = " << *regPtr3 << endl;

// *regPtr3 = 50;

// ERROR: Cannot modify the pointed value because int is const.

// regPtr3 = &dataReg;

// ERROR: Cannot change the address because the pointer itself is const.

return 0;

}

```

Output:

```

<terminated> (exit value: 0) Assingment.exe [C/C++
--- regPtr1 : const int* ---
Status Register = 177
--- regPtr2 : int* const ---
Control Register = 25
--- regPtr3 : const int* const ---
Status Register = 177

```

Problem 4 — Calibration Packet Parser

Source code:

```
#include<iostream>

using namespace std;

bool parsePacket(const int* rawData, int size,int** outMin, int** outMax){
    if(size <= 0){
        return false;
    }
    const int* minPtr = rawData;
    const int* maxPtr = rawData;
    for(int i = 1; i < size; i++){
        if(*(rawData + i) < *minPtr){
            minPtr = rawData + i;
        }
        if(*(rawData + i) > *maxPtr){
            maxPtr = rawData + i;
        }
    }
    *outMin = const_cast<int*>(minPtr);
    *outMax = const_cast<int*>(maxPtr);
    return true;
}

int main(){
    int packet[] = {45, 12, 67, 8, 55, 31};
    int* minPtr = nullptr;
    int* maxPtr = nullptr;
    if (parsePacket(packet, 6, &minPtr, &maxPtr)) {
        cout << "Calibration Min : " << *minPtr << endl;
        cout << "Calibration Max : " << *maxPtr << endl;
    }
```

```
    }  
  
    return 0;  
  
}
```

Output:

```
Calibration Min : 8  
Calibration Max : 67
```

Problem 5 — Drone Navigation Utilities

Source code:

```
#include<iostream>  
  
#include<cmath>  
  
using namespace std;  
  
inline double distanceBetween(double x1, double y1, double x2, double y2)  
{  
    return sqrt(pow(x2-x1, 2) + pow(y2-y1, 2));  
}  
  
inline double toRadians(double degrees)  
{  
    return degrees * (M_PI / 180.0);  
}  
  
inline double clamp(double value, double minVal, double maxVal)  
{  
    if(value < minVal)  
        return minVal;  
    if(value > maxVal)  
        return maxVal;  
    return value;  
}  
  
inline bool isInSafeZone(double x, double y, double cx, double cy, double radius)  
{
```

```

    return distanceBetween(x, y, cx, cy) <= radius;
}

int main(){
    double homeX = 0.0;
    double homeY = 0.0;
    double safeRadius = 50.0;

    // Waypoint 1
    double x1 = 30.0;
    double y1 = 40.0;

    // Waypoint 2
    double x2 = 60.0;
    double y2 = 10.0;

    // Waypoint 3
    double x3 = -20.0;
    double y3 = 15.0;

    cout << "--- Drone Navigation ---" << endl;

    double distance1 = distanceBetween(homeX, homeY, x1, y1);
    cout << "Waypoint 1 Distance : " << distance1 << endl;

    cout << "Waypoint 1 Safe Zone : " << (isInSafeZone(x1, y1, homeX, homeY, safeRadius)?
    "Yes" : "No") << endl;

    double distance2 = distanceBetween(homeX, homeY, x2, y2);
    cout << "Waypoint 2 Distance : " << distance2 << endl;

    cout << "Waypoint 2 Safe Zone : " << (isInSafeZone(x2, y2, homeX, homeY,
    safeRadius)? "Yes" : "No") << endl;

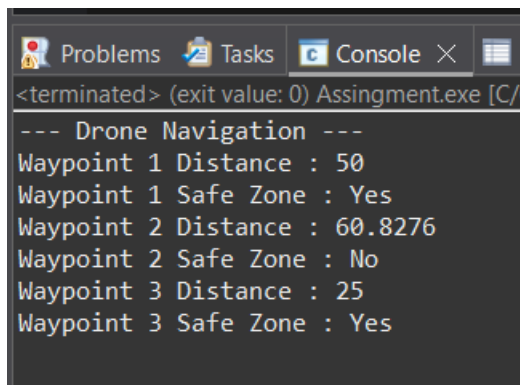
    double distance3 = distanceBetween(homeX, homeY, x3, y3);
    cout << "Waypoint 3 Distance : " << distance3 << endl;

    cout << "Waypoint 3 Safe Zone : " << (isInSafeZone(x3, y3, homeX, homeY, safeRadius)
    ? "Yes" : "No") << endl;

    return 0;
}

```

Output:

A screenshot of a console window with a dark background. The window has tabs for 'Problems', 'Tasks', and 'Console'. The console text shows the program has terminated with an exit value of 0. The output is as follows:

```
<terminated> (exit value: 0) Assingment.exe [C/
--- Drone Navigation ---
Waypoint 1 Distance : 50
Waypoint 1 Safe Zone : Yes
Waypoint 2 Distance : 60.8276
Waypoint 2 Safe Zone : No
Waypoint 3 Distance : 25
Waypoint 3 Safe Zone : Yes
```

Question 3 — HR Payroll Engine

Source code:

```
#include<iostream>

#include<string>

#include<iomanip>

using namespace std;

class Employee{
private:
    int empId;
    string name;
    string department;
    char grade;
    double basicSalary;
    bool isActive;
    static int employeeCount;
public:
    // Constructor
    Employee(){
        empId = 1001 + employeeCount;
        employeeCount++;
        name = "";
```

```

    department = "";
    grade = 'D';
    basicSalary = 0;
    isActive = true;
}

// Setters

void setName(const string& n){
    if(n.empty()){
        cout << "ERROR: Name cannot be empty." << endl;
    }
    else{
        name = n;
    }
}

void setDepartment(const string& dept){
    if(dept == "Engineering" ||
        dept == "HR" ||
        dept == "Finance" ||
        dept == "Operations"){
        department = dept;
    }
    else{
        cout << "ERROR: " << dept
            << " is not a registered department." << endl;
    }
}

void setGrade(char g){
    if(g == 'A' ||
        g == 'B' ||

```

```

        g == 'C' ||
        g == 'D'){
            grade = g;
        }
    else{
        cout << "ERROR: Invalid grade " << g
            << ". Accepted values: A, B, C, D."
            << endl;
    }
}

void setBasicSalary(double salary){
    if(salary > 10000 && salary < 500000){
        basicSalary = salary;
    }
    else{
        cout << "ERROR: Salary must be between "
            << "Rs.10,000 and Rs.5,00,000. "
            << "Value rejected." << endl;
    }
}

void deactivate(){
    isActive = false;
}

// Getters

int getEmpId() const{
    return empId;
}

string getName() const{
    return name;
}

```

```
}  
  
string getDepartment() const{  
    return department;  
}  
  
char getGrade() const{  
    return grade;  
}  
  
double getBasicSalary() const{  
    return basicSalary;  
}  
  
bool getIsActive() const{  
    return isActive;  
}  
  
// Business Logic  
  
double computeAllowances() const{  
    if(grade == 'A')  
        return basicSalary * 0.40;  
    else if(grade == 'B')  
        return basicSalary * 0.30;  
    else if(grade == 'C')  
        return basicSalary * 0.20;  
    else  
        return basicSalary * 0.10;  
}  
  
double computeGrossSalary() const{  
    return basicSalary + computeAllowances();  
}  
  
double computeTax() const{  
    double gross = computeGrossSalary();
```

```

    if(gross <= 50000){
        return 0;
    }
    else if(gross <= 100000){
        return (gross - 50000) * 0.10;
    }
    else{
        return 5000 + (gross - 100000) * 0.20;
    }
}

double computeNetSalary() const{
    return computeGrossSalary() - computeTax();
}

void printPayslip() const{
    cout << "===== " << endl;
    cout << " EMPLOYEE PAYSLIP - AUG 2026" << endl;
    cout << "===== " << endl;
    cout << "Emp ID : " << empId << endl;
    cout << "Name : " << name << endl;
    cout << "Department : " << department << endl;
    cout << "Grade : " << grade << endl;
    cout << "Status : "
        << (isActive ? "Active" : "Inactive")
        << endl;
    cout << "----- " << endl;
    cout << fixed << setprecision(2);
    cout << "Basic Salary : Rs. "
        << basicSalary << endl;
    cout << "Allowances ("

```

```

        << (grade == 'A' ? 40 :
            grade == 'B' ? 30 :
            grade == 'C' ? 20 : 10)
        << "%) : Rs. "
        << computeAllowances() << endl;
    cout << "Gross Salary : Rs. "
        << computeGrossSalary() << endl;
    cout << "-----" << endl;
    cout << "Tax Deduction : Rs. "
        << computeTax() << endl;
    cout << "Net Salary : Rs. "
        << computeNetSalary() << endl;
    cout << "===== " << endl;
}

static int getEmployeeCount(){
    return employeeCount;
}

// Accept Details
void acceptDetails(){
    string nameInput;
    string deptInput;
    char gradeInput;
    double salaryInput;
    cout << "\nEnter name: ";
    getline(cin >> ws, nameInput);
    setName(nameInput);
    cout << "Enter department: ";
    getline(cin >> ws, deptInput);
    setDepartment(deptInput);

```

```

        cout << "Enter grade: ";
        cin >> gradeInput;
        setGrade(gradeInput);
        cout << "Enter basic salary: ";
        cin >> salaryInput;
        setBasicSalary(salaryInput);
    }
};

// Static member definition
int Employee::employeeCount = 0;

int main(){
    // One object on stack
    Employee e1;
    // Two objects on heap
    Employee* e2 = new Employee();
    Employee* e3 = new Employee();
    // Accept details
    e1.acceptDetails();
    e2->acceptDetails();
    e3->acceptDetails();
    // These produce compiler errors because
    // empId and basicSalary are private members.
    // e1.empId = 999;
    // ERROR: empId is private and cannot be accessed directly.
    // e1.basicSalary = -1000;
    // ERROR: basicSalary is private and cannot be accessed directly.
    // Use setBasicSalary() so validation is applied.
    // Print payslips
    e1.printPayslip();

```

```

e2->printPayslip();
e3->printPayslip();
// Simulate resignation
e3->deactivate();
if(!e3->getIsActive()){
    cout << e3->getName()
        << " is no longer active. Payroll skipped."
        << endl;
}
cout << "Total Employees : "
    << Employee::getEmployeeCount()
    << endl;
// Free heap memory
delete e2;
delete e3;
return 0;
}

```

Output:

```

=====
EMPLOYEE PAYSIP - AUG 2026
=====
Emp ID : 1001
Name : Pranav Bagal
Department : Engineering
Grade : A
Status : Active
-----
Basic Salary : Rs. 60000.00
Allowances (40%) : Rs. 24000.00
Gross Salary : Rs. 84000.00
-----
Tax Deduction : Rs. 3400.00
Net Salary : Rs. 80600.00
=====

```

Bonus — Struct Padding

Source code:

```
#include <iostream>

using namespace std;

struct Layout1 {
    char c1;
    int i;
    char c2;
};

struct Layout2 {
    int i;
    char c1;
    char c2;
};

int main() {
    cout << "Size of Layout1 : " << sizeof(Layout1) << " bytes" << endl;
    cout << "Size of Layout2 : " << sizeof(Layout2) << " bytes" << endl;
    /*
```

STRUCT PADDING:

1. Why can the sizes differ?

The members of a struct are stored in memory in the same order in which they are declared.

Layout1:

char c1 -> 1 byte

padding -> 3 bytes

int i -> 4 bytes

char c2 -> 1 byte
padding -> 3 bytes

So Layout1 usually takes 12 bytes.

Layout2:

int i -> 4 bytes
char c1 -> 1 byte
char c2 -> 1 byte
padding -> 2 bytes

So Layout2 usually takes 8 bytes.

2. What is padding?

Padding means extra unused bytes added by the compiler between or after members of a struct.

The compiler adds padding so that data members are placed at memory addresses suitable for their alignment requirements.

For example, an int usually requires 4-byte alignment.

3. Why does member order matter?

The order of members can change the amount of padding required and therefore can change the total size of the struct.

This is especially important when defining:

- Network packet headers
- Hardware register maps
- Communication protocols
- Memory-mapped structures

In such cases, the exact memory layout may be important.

Incorrect padding or member ordering can cause the program to read data from the wrong memory location.

```
*/
```

```
return 0;
```

```
}
```

Output:

Size of Layout1 : 12 bytes

Size of Layout2 : 8 bytes